Incident Audit: Token-Burn & Verifier Investigation

SQL_TEST_01 — Ask-Your-Database Agent (Builds 6/7) — 2026-08-06

Scope: everything from the token/latency/tool-use tracking request through full browser-confirmed resolution, same working day.

Timezone: all times below are Pacific (UTC-7), matching the git commit history and local machine clock. Raw application logs (`logs/tool_calls.jsonl`) are recorded in UTC by the container; every timestamp in this report has been converted (UTC − 7h) for consistency.

Evidence basis: every timestamp and token count below is pulled directly from `logs/tool_calls.jsonl`, git commit history, or command output captured live during the investigation — not reconstructed from memory. Where a stretch of work (prompt edits, code review, diagnosis) produced no independently-timestamped log entry, this report says so explicitly rather than inventing a time for it.

Executive summary

A routine feature request — add token count to the UI alongside the existing latency/tool-use display — surfaced, in order: a verifier false-positive, a 36.5-second latency regression, and a single question that cost **177,090 tokens** and returned a **wrong answer**. Chasing the wrong-answer bug through four iterations of prompt engineering repeatedly fixed one symptom while exposing another, until the user identified the actual root cause: the source data stored every checkout as one unstructured, delimited text blob, and every fix so far had been the agent (and the verifier) compensating for that at query time, on every question, instead of the data being modeled correctly once. Normalizing the data (a new `clean.allocation_items` table, one row per item, built once offline) resolved every open symptom at once — token cost, correctness, and the verifier's blind spots — without needing further prompt patches. Final state, browser-confirmed: the original 177,090-token/wrong-answer case now costs **8,024 tokens** and answers correctly (a 95.5% reduction), and three other categories tested for the first time in the browser (laptops, microphones, accessories) all also answer correctly and cheaply on the first try.

Timeline

Phase 0 — Feature ships, first blind spot surfaces

11:28 – 11:41 PDT

TIME	EVENT	
11:28:20	Commit `eee58cb` — token usage tracking added to `agent.py` / `logging_utils.py` / `app.py`, alongside the existing latency and tool-use display. (This followed a plan change mid-request: the user's first instruction was answer-only, no logging; corrected almost immediately to "include logs, refactor.")	
11:39:26	"What can you do?" — first live post-deploy question	49908cb5e5d6
11:39:58	"What's in the database?"	4674e8df6d97
11:41:22	"How many cameras do you have?" — **68,853 input tokens**, verifier: `Unverified`	edbce40d1a73

> **Finding:** the verifier's own reasoning claimed a "critical contradiction" in a correctly-answered question — a false positive. The user paused to ask directly whether this was really the first genuine test of the verifier working, rather than accepting the red badge at face value.

Phase 0.5 — Verifier self-correction, MAX_FIELD_CHARS, eval-harness stabilization

`reconstructed — bounded by real timestamps, not individually logged`

This stretch of work (prompt diagnosis, code edits, review) produced no individually-timestamped log entries of its own — the evidence for it is the eval/verification log entries at its start and end (11:41 PDT above, 13:32 PDT below), plus the resulting code changes.

- **User: "Yes, investigate."** First diagnosis attempted a fix (teaching the verifier that a sum exceeding a total isn't automatically a contradiction) — this did not fully hold up under closer review.
- **Self-correction, not user-prompted:** re-reading the full evidence block (not just the last tool call) found 2 of 5 per-category counts individually *exceeded* the stricter total they were being compared against — a sign the categories were measured under a broader SQL condition than the total, not a benign overlap. Reported this directly to the user as a walk-back of the first diagnosis.
- `VERIFIER_SYSTEM_PROMPT` **fixed** to check whether a per-category condition is a true logical subset of a stricter total's condition, not just compare the resulting numbers.
- **2 new permanent regression cases** added to `verify_eval_cases.py` — the real camera false-positive case, and a genuine-overlap look-alike (laptop/tablet) that must still pass — both stable across 10/10 repeated runs.
- **User: "How do we stop the token burn?"** → "Review the other token counts logged today for similar pattern." A historical sweep of the day's logged tool-call sizes found the same "large single result" shape recurring. `MAX_FIELD_CHARS = 500` added to `tools.py`, capping any individual field value (not row count, which was already capped) — verified a GROUP BY result drop from 18,751→9,456 chars and a `SELECT *` drop from 55,504→12,051 chars.
- **Eval-harness flakiness ("dig in and resolve"):** 2 flaky eval cases fixed (one reworded to a more directive question, one widened to accept a list of legitimate tool choices via `run_eval.py` supporting `expected_tool` as a list) — plus an unrelated `UnicodeEncodeError` **crash** found incidentally during repeated stress-testing, fixed with a UTF-8 stdout reconfigure in both `agent.py` and `verify_answer.py`.

Phase 1 — Latency anomaly surfaces

`13:32 PDT`

`13:32:58` "How many cameras do you have?" — 10,375 in / 482 out = **10,857 tokens** (normal), but `search_summaries` latency = **36,468.83ms** `32506ed31cb4`

User pasted the real browser transcript (10,857 tokens, 36,469ms shown against `search_summaries`) and asked to confirm whether the token count was too high. It wasn't — it matched the day's per-turn baseline — but the 36.5-second latency was a real, separate anomaly, flagged and then investigated on explicit instruction ("Investigate").

Phase 2 — Latency root-cause investigation

`~13:35 – 14:05 PDT`

Hypothesis 1 (disproven): Streamlit's dev-mode file watcher, which walks `transformers`' optional vision submodules and produces a harmless-looking `torchvision` `ModuleNotFoundError` — timestamped inside the exact 36-second window. Implemented `.streamlit/config.toml` (`fileWatcherType = "none"`), rebuilt, and tested with a **non-Streamlit** process inside the container: still 36,198.6ms. Disproven by direct test, not assumption. (The config change itself was kept — it's still a legitimate fix for the documented log-noise issue, just not the latency cause.)

Hypothesis 2 (confirmed): the container had no persistent volume for the HuggingFace model cache — every fresh container re-downloads the ~90MB `all-MiniLM-L6-v2` model from HuggingFace Hub on its first embedding call. Confirmed directly: the cache directory didn't exist before the test call, and a second fresh process after the cache was warm measured 2,391.0ms.

User asked what it would take for the agent to drive the browser itself (a genuine capability question) — answer: a browser-automation MCP server (e.g. Playwright) with real browser binaries, which isn't configured in this environment; `WebFetch` can't click/type/wait on a reactive Streamlit session. This kept browser confirmation a manual, user-driven step throughout the rest of the incident, consistent with the project's standing rule.

Decision (user-selected, "option 2"): bake the model into the Docker image at build time (a `RUN` step in the `Dockerfile`) rather than a runtime-mounted cache volume — no runtime network dependency at all, at the cost of a longer image build. Implemented, rebuilt (57.9s at *build* time, not runtime), force-recreated. **Verified:** a genuinely fresh, just-recreated container's first embedding call measured **2,690.1ms** (vs. the original 36,468.83ms).

Phase 3 — The real token-burn incident

14:19 PDT

14:19:00 "How many different types of cameras do you have?" — 9 tool calls, 174,605 in / 2,485 out = **177,090 tokens** 98ec17db8b97

> **This time the verifier was right, not wrong.** `Unverified (correctly)` — the proposed answer mixed per-model counts (98/96/89/102) pulled under one SQL `WHERE` filter with a different count (207) pulled under a broader filter, presenting them as one consistent measurement. Same underlying skill the verifier had just been taught in Phase 0.5, now catching a genuine defect instead of a phantom one.

Root cause traced directly from the log: turn 4 ran `SELECT DISTINCT summary ... LIMIT 50`, dumping 18,230 characters of raw, unaggregated checkout-bundle text instead of an aggregate query — and because the Anthropic API is stateless, every one of the 9 turns re-sent the full growing history of the round. On top of that, Build 6's tiered memory (`MAX_FULL_FIDELITY_ROUNDS = 3`) meant this entire round would replay in full into the next 3 questions' context too, not just this one.

The screenshot below (saved by the user at 15:29:43 PDT, capturing this same result) documents the verifier's exact reasoning and the 177,090-token total from this question.

Total: 5 different camera types tracked in the system across 350 allocations (some allocations contain multiple cameras).

⚠ Unverified: The proposed answer claims there are 5 different camera types and provides specific allocation counts for each. However, the evidence shows a critical discrepancy: when the SQL query explicitly checks for each camera type individually across ALL rows in the allocations table (not filtered by '%CAMERA%'), the counts are: INSTA360 X4 CAMERA (207), NIKON Z6 III MIRRORLESS BODY (199), PANASONIC S5 II BODY (216), FUJIFILM X-T5 BODY (214), and OLYMPUS OM-D E-M1 BODY (228). The proposed answer uses different numbers (98, 96, 89, 102 for the non-Insta360 cameras) that come from a query filtered to WHERE summary ILIKE '%CAMERA%' - a narrower subset. The proposed answer conflates these different measurements without clarifying that the INSTA360 count (207) and the other camera counts (98, 96, 89, 102) are from different query conditions. Most critically, the broader query shows NIKON count as 199 (not 98), PANASONIC as 216 (not 96), FUJIFILM as 214 (not 89), and OLYMPUS as 228 (not 102) - indicating these narrower counts represent a much smaller subset. The answer is misleading by presenting mismatched category counts as if they belong to the same measurement.

› Tools used (9) · 177,090 tokens (174,605 in / 2,485 out)

Phase 4 — Four fix iterations, each tested and honestly reported `14:38 – 15:00 PDT`

v1 — steer toward one aggregate query `14:38:19`

`2b5ba182c7da`		11,288 in / 638 out = **11,926 tokens** (93% reduction)	`Unverified (correctly)`

Cheap, but wrong: the new single query mislabeled "NIKON Z6 III CAMERA CASE" as a camera type, and didn't strip the allocation-level "Returned: " prefix, creating a spurious duplicate "type." The verifier caught this correctly too.

User: "Yes! … there are other such scenarios that exist. Audit the summary column for such other possibilities." A full audit parsed every `summary` value directly in Python (bypassing keyword-based bias entirely) and found the closed universe: **55 distinct item names**. Confirmed the accessory/category-keyword trap recurs across at least 7 other product families (projector/cable, light/stand, laptop/case+charger, mic/battery, tripod/case, recorder/case, drawing-tablet/pen) — plus a second, sneakier bug: real cameras are inconsistently named ("... BODY"/"... MIRRORLESS" for 4 of 5 brands, only INSTA360 literally says "CAMERA"), so a keyword search both over- and under-counts at once.

v2 — generalized accessory/naming guidance `14:50 – 14:51`

`4096e3913ff4`	camera	58,232 in / 2,892 out = **61,124 tokens**	`Verified – correct (5 types incl. Fujifilm)`
`ddce9ba7b2e8`	laptop	12,565 in / 750 out = **13,315 tokens**	`Verified – correct`

Camera finally correct — but at 61,124 tokens, worse than v1's (wrong) 11,926. Making the model "pull the full list when ambiguous" as a soft suggestion caused more exploratory back-and-forth, not less.

v3 — mandatory full-list-first rule `14:59 – 15:00`

`955e09204a29`	camera	22,471 in / 1,450 out = **23,921 tokens**	`Verified – correct`
`7e40945aa1fe`	laptop	44,066 in / 1,403 out = **45,469 tokens**	`Verified – correct, but 3.4x more expensive than v2's laptop run`

`fd8ed1d3fad1`	microphone	14,115 in / 686 out = **14,801 tokens**	`Unverified — agent was RIGHT, verifier was WRONG`

New failure mode: making the full-list step unconditional fixed camera's correctness but taxed laptop (which never needed it) with a 3.4x cost increase. Worse: the microphone answer correctly excluded "RODE MIC BATTERY" as an accessory — exactly the judgment the prompt now teaches — but the verifier flagged it unsupported anyway, calling the exclusion "an editorial judgment not evident in the tool evidence." A verifier false *negative* this time: the mirror image of Phase 0's false positive, caused by the verifier never having received the same domain knowledge the agent's prompt now had.

Phase 5 — Root-cause fix: normalize the data

`~15:00 – 15:16 PDT`

> **User's diagnosis, verbatim in substance:** "If the data was cleaned and transformed fully these issues will not occur... the agent/model is working harder than it needs to due to the data naming convention inconsistencies." Confirmed correct — every fix in Phase 0.5 and Phase 4 had been the agent (and verifier) compensating at query time for the same underlying problem: no structured `item_name` / `category` / `is_accessory` / `is_returned` columns existed at all.

- `migrations/013_allocation_items_schema.sql` — new `clean.allocation_items` table, one row per item.
- `src/build_allocation_items.py` — ETL script with a closed, audited `ITEM_CLASSIFICATION` lookup for all 55 known item names; fails loudly on any unclassified name rather than silently miscategorizing. Run: **14,347 item rows loaded from 1,535 allocations, zero unclassified names.**
- `.github/workflows/ci.yml` updated in **both** jobs (migration list + a new "Build normalized allocation items" step) — deliberately checked both places, since a prior incident this same week (commit `08b807b`) was exactly a migration missing from one of two duplicated CI lists.
- `SYSTEM_PROMPT` **shrank**, rather than grew, once it could point at resolved facts instead of teaching keyword heuristics.

v4 — point the agent at the normalized table `15:16 – 15:17`

`f1683cc1d48c`	camera	7,377 in / 302 out = **7,679 tokens**	**Verified — correct AND cheap**
`e0097d958349`	laptop	13,338 in / 343 out = **13,681 tokens**	**Verified — but WRONG (1,501, forgot `is_accessory` filter)**
`1527035daac7`	microphone	4,588 in / 169 out = **4,757 tokens**	**Verified — but WRONG (2 types, counted the battery)**

> The new table's data was correct, but the prompt showed the `is_accessory = false` filter as one worked example (camera), not a stated rule — the model applied it to camera but dropped it for laptop and microphone. The verifier passed both wrong answers, because both were internally consistent with their own (wrong) SQL — the same blind spot as before, one level downstream.

v5 — final: default rule, not an example `15:20 PDT`

`26fc383faaaa`	camera	**8,753 tokens**	`Correct`
`b6de9284effb`	laptop	**7,859 tokens**	`Correct — 482`
`992486b0a2be`	microphone	**7,944 tokens**	`Correct — 1`
`c69fca33055a`	tripod (untested category)	**7,857 tokens**	`Correct — 245, first-try, no further prompt changes needed`

Phase 6 — Regression check & browser confirmation

`15:20 – 15:43 PDT`

- Eval suite: 6/7 → 7/7 → 7/7 across three runs. The one failure was confirmed non-systematic by re-running that exact question standalone and getting the expected tool choice back — pre-existing tool-choice non-determinism, not a regression from today's changes.
- **15:34:56 PDT** — checked `agent_scratch.chat_rounds` before recommending a browser test: found 3 persisted rounds, *all from before today's fixes*, including the original 177,090-token round. Since Build 6's tiered memory replays the last 3 rounds in full into every new question, the very next browser question would have re-inherited that cost regardless of whether the fix worked. Truncated the table (chat history only — no project data touched) before testing.
- **15:40:51 – 15:43:09 PDT — browser-confirmed, live, by the user:**

`6c2f9c45d3b9`	camera	**8,024 tokens**	`5 types ✓`
`c5c66ded6b00`	laptop	**6,018 tokens**	`482 ✓`
`13d4b9704d14`	microphone	**7,434 tokens**	`1 type ✓`
`f7613e03fc17`	accessories	**10,221 tokens**	`5,762 ✓ (independently confirmed: sums exactly to the total of every accessory item's count in the audit)`

Token savings — before vs. after

QUESTION	BEFORE	AFTER	REDUCTION	CORRECTNESS CHANGE
"How many different types of cameras do you have?" (the incident question)	177,090	8,024	95.5%	Wrong (4 types) → Correct (5 types)
"How many laptops do you have?"	45,469 (v3, still wrong at 1,501 by v4)	6,018	86.8%	Wrong (1,501) → Correct (482)
"How many types of microphones do you have?"	14,801	7,434	49.8%	Correct answer, but a false-negative verifier badge → Correct + verified
"How many tripods do you have?" (never asked before the fix)	—	7,857	—	Correct on first try, no category-specific tuning needed

"Before" figures use each question's first attempt after the underlying bug was reproducible, not necessarily the single worst run recorded — the incident question's 177,090 tokens is the actual value from the original failing round.

Decision log

DECISION	CHOSEN OVER	WHY
Bake the embedding model into the Docker image at build time	A runtime-mounted persistent cache volume	User's explicit choice ("option 2") — zero runtime network dependency on any container start, not just after the first one.
Normalize checkout items into a new table	Continuing to patch `SYSTEM_PROMPT` / `VERIFIER_SYSTEM_PROMPT`	User's diagnosis: four prompt iterations kept trading one symptom for another because the root problem was in the data, not the prompt. One offline ETL pass fixed cost, correctness, and the verifier's blind spot simultaneously.

State the accessory-exclusion filter as a default rule, not a worked example	Leaving it as an example the model could generalize from	v4 showed the model applies a worked example inconsistently across categories; a stated default rule (v5) generalized correctly to an entirely untested category (tripod) on the first try.
Truncate `agent_scratch.chat_rounds` before browser testing	Testing on top of existing history	All 3 persisted rounds predated the fix; Build 6's tiered memory would have replayed the original 177,090-token round into every new question's context regardless of whether the fix itself worked.

Verifier: two distinct failure modes found, both fixed differently

MODE	EXAMPLE	FIX
False positive — flagged a correct answer as unsupported	Phase 0 (11:41 PDT): per-category counts under different SQL conditions looked contradictory by number alone	`VERIFIER_SYSTEM_PROMPT` taught to check condition subset-ness, not just compare numbers
False negative — passed a wrong answer as supported	Phase 4 v3 (15:00 PDT) and Phase 5 v4 (15:16-15:17 PDT): the verifier can only check consistency between an answer and its own evidence, not whether that evidence itself used the right filter	Fixed indirectly — once the underlying data was normalized and the query became simple and directly correct, there was nothing left for the verifier to be blind to. No further verifier prompt changes were needed after Phase 5.

Files and artifacts changed

- `src/agent.py` — token usage tracking, stdout UTF-8 fix, `SYSTEM_PROMPT` rewritten twice (compensating rules added in Phase 4, then replaced with a short pointer to the normalized table in Phase 5)

- `src/tools.py` — `MAX_FIELD_CHARS` cap
- `src/verify_answer.py` — `VERIFIER_SYSTEM_PROMPT` subset-condition check, stdout UTF-8 fix
- `src/verify_eval_cases.py` — 2 new permanent regression cases
- `src/eval_cases.py`, `src/run_eval.py` — 2 flaky cases fixed, list-valued `expected_tool` support
- `src/logging_utils.py`, `src/app.py` — usage display, already committed earlier the same day (`eee58cb`)
- `migrations/013_allocation_items_schema.sql`, `src/build_allocation_items.py` — new normalized table + ETL (new)
- `.github/workflows/ci.yml` — migration + build step added to both jobs (new)
- `Dockerfile` — model baked in at build time, `.streamlit/config.toml` copied in
- `.streamlit/config.toml` — file watcher disabled (new)

As of this report, this batch is fully tested and browser-confirmed but not yet committed to git.

Lessons

- A verifier catching a real defect and a verifier producing a false positive can look identical from the outside (a red badge) — both require reading the actual evidence before accepting or dismissing the flag.
- A prompt-level fix that removes one failure mode can introduce a new one in the same turn (v1's cost win came with a new correctness bug; v3's correctness fix came with a new cost regression and a new verifier blind spot). Each iteration in this incident was tested and honestly reported before being called "done," including the ones that didn't work.
- Compensating for messy data at inference time is a recurring, per-question cost; fixing the data once is a one-time cost. The fourth prompt iteration would likely not have been the last one — the fifth data-layer fix was.
- Persisted conversation history can silently reintroduce a just-fixed bug's cost into the very next test, if verification doesn't check what's actually sitting in that history first.